

Giới hạn tính toán và hệ thống (Compute and Systems Limits)

PHỤ THUỘC B/10-thiet-ke-kien-truc (bốn thước đo chi phí — chương này biến cột “độ trễ” ở đó thành con số); B/03-giai-phau-he-thong (ràng buộc cứng); hữu ích nhưng không bắt buộc: D/19-numpy-toi-pytorch nếu bạn muốn tự đo.

TÓM TẮT

Chương này trả lời một câu hỏi rất cụ thể: vì sao bạn giảm FLOPs một nửa mà mô hình không nhanh hơn. Hầu hết phép toán trong một mạng thị giác không chờ đơn vị tính toán; chúng chờ bộ nhớ. Đại lượng phân biệt hai chế độ đó là cường độ số học, tức số FLOP làm được trên mỗi byte phải chuyển. Nó tính được bằng giấy bút trong hai phút. Đọc xong bạn ước lượng được bộ nhớ và độ trễ trước khi viết code. Bạn cũng đọc được một bảng “FLOPs thấp hơn ba lần” mà không bị nó dẫn sai hướng, và chọn đúng biện pháp cho đúng chế độ nghẽn thay vì thử tất cả. Nếu chỉ nhớ một điều: FLOPs là thuộc tính của mô hình, còn thời gian là thuộc tính của cặp mô hình–máy. Cây cầu giữa chúng là cường độ số học đem so với điểm gãy roofline của chính cỗ máy bạn chạy: khoảng 150 FLOP/byte trên A100, khoảng 330 trên Jetson Orin.

KIẾN THỨC CẦN CÓ TRƯỚC

- **FLOPs** — số phép nhân–cộng dấu phẩy động cần cho một lần chạy mô hình. Nó đo *khối lượng công việc tính toán*, không đo thời gian. Phân biệt hai thứ này là toàn bộ nội dung của chương. Có một nhập nhằng phổ biến: nhiều bài báo ghi “FLOPs” nhưng thực ra đếm số phép nhân–cộng, tức một nửa con số theo định nghĩa chặt.
- **băng thông bộ nhớ (memory bandwidth)** — số byte mỗi giây chuyển được giữa bộ nhớ chính của thiết bị và chip tính toán, ví dụ 2000 GB/s trên một GPU máy chủ và khoảng 200 GB/s trên một mô-đun nhúng. Đây là một hằng số vật lý của máy, không phải thứ tối ưu phần mềm thay đổi được.
- **cường độ số học (arithmetic intensity)** — tỉ số giữa số FLOP thực hiện và số byte phải chuyển để thực hiện chúng. Nó trả lời câu “mỗi byte kéo về được dùng để làm bao nhiêu việc”, và so nó với một hằng số của máy là ra ngay bạn đang bị giới hạn bởi tính toán hay bởi bộ nhớ.
- **độ trễ (latency)** — thời gian từ lúc một đầu vào đi vào hệ thống tới lúc có kết quả cho *chính* đầu vào đó. Đây là đại lượng mà hệ thống thời gian thực quan tâm.
- **thông lượng (throughput)** — số mẫu xử lý xong trên một đơn vị thời gian khi được phép gom nhiều mẫu lại cùng lúc. Hai đại lượng này xung đột nhau: gom mẫu làm thông lượng tăng và độ trễ xấu đi.
- **kích hoạt (activation)** — các tensor trung gian sinh ra ở giữa mạng trong *lượt xuôi* (một lần chạy mô hình từ đầu vào tới đầu ra). *Lượt ngược* là lần chạy tính gradient theo chiều ngược lại, do thuật toán lan truyền ngược điều khiển. Nó cần đúng các tensor đó, nên khi huấn luyện ta phải giữ lại tất cả. Vì vậy kích hoạt là phần bộ nhớ tỉ lệ với số mẫu trong một batch và với độ phân giải ảnh, khác hẳn tham số vốn cố định.
- **kernel (trong ngữ cảnh GPU)** — một chương trình nhỏ chạy trên GPU thực hiện một phép toán, ví dụ “một tầng convolution” hoặc “một phép cộng tensor”. CPU phải *phóng* từng kernel và mỗi lần phóng tốn một khoản thời gian cố định. Đừng nhầm với “kernel” nghĩa của số convolution ở chương 10 — hai nghĩa hoàn toàn khác nhau.

- **GPU utilization và MFU** — “utilization” chỉ cho biết GPU có đang bận hay không, kể cả khi nó bận chờ bộ nhớ; MFU (*model FLOPs utilization*) là tỉ lệ giữa FLOPs thật sự làm được và đỉnh lý thuyết của chip. Một GPU có thể util 95% mà MFU chỉ 10%, và chính khoảng cách giữa hai con số đó là manh mối chẩn đoán mạnh nhất bạn có.

CÂU HỎI MỞ ĐẦU

- Q1. Hai mô hình cùng số FLOPs, vì sao một cái chạy nhanh gấp năm lần cái kia trên cùng một GPU?
- Q2. Vì sao mô hình vừa khít bộ nhớ khi suy luận lại tràn khi huấn luyện, dù trọng số không đổi một byte nào?
- Q3. Điểm gây roofline của một mô-đun nhúng *cao gấp đôi* của một GPU máy chủ — điều đó nói gì về danh sách tối ưu bạn mang từ máy chủ xuống thiết bị?
- Q4. Bạn tối ưu mô hình suốt một tuần và GPU vẫn nhàn rỗi bốn mươi phần trăm thời gian. Vì sao lỗi có thể hoàn toàn không nằm trong mô hình?
- Q5. Bạn đo được lượt xuôi của một mạng lớn mất 0,2 ms. Vì sao con số đó gần như chắc chắn là đối trá, và đối trá theo cơ chế nào?

1.1 Đặt vấn đề

Bạn thay một khối trong mô hình bằng một khối rẻ hơn một bậc về **FLOPs**, đo lại, và độ trễ không nhúc nhích — đôi khi còn tệ hơn. Đây không phải chuyện hiếm và cũng không phải lỗi cài đặt: nó là hệ quả trực tiếp của một sự thật vật lý mà bảng so sánh trong paper không bao giờ nhắc tới. Bộ xử lý hiện đại *tính* nhanh hơn khả năng *nạp dữ liệu* của nó hàng trăm lần, nên phần lớn các phép toán trong một mạng nơ-ron không chờ đơn vị tính toán mà chờ bộ nhớ. Khi bạn đang bị giới hạn bởi băng thông mà lại đi tối ưu số phép nhân, bạn đang tối ưu đúng cách nhưng nhầm chỗ.

Đọc nhanh, nếu bạn chỉ có năm phút. Có một đại lượng duy nhất quyết định bạn đang bị giới hạn bởi cái gì: **cường độ số học** (*arithmetic intensity*) — số phép tính thực hiện được trên mỗi byte phải chuyển qua bộ nhớ. Đem so nó với một hằng số của phần cứng (**điểm gây** (*ridge point*) = đỉnh tính toán chia cho băng thông) là ra ngay chế độ nghẽn, và mỗi chế độ có một tập biện pháp riêng mà các tập kia hoàn toàn vô ích. Trên Jetson AGX Orin, điểm gây nằm quanh 330 FLOP/byte, nghĩa là gần như mọi thứ trừ *convolution* dày và *GEMM* lớn đều nghẽn bộ nhớ.

Giá trị của chương này rất cụ thể. Bạn dự đoán được *trước khi viết một dòng code* rằng một tối ưu sẽ giúp hay không. Bạn ước lượng được bộ nhớ và độ trễ bằng giấy bút với sai số một bậc. Và bạn đọc được một bảng “FLOPs thấp hơn ba lần” mà không bị nó dẫn đi sai hướng. Nếu bạn đã từng đo latency của một pipeline SLAM trên thiết bị nhúng, chương này chủ yếu là đặt tên và công thức hoá những thứ bạn đã cảm nhận bằng tay.

TRỰC GIÁC

Hãy hình dung GPU như một nhà máy khổng lồ với một cửa nhập hàng rất hẹp. Bên trong có hàng nghìn thợ (**CUDA core** và **lõi tensor** (*tensor core*)) làm việc nhanh kinh khủng; nhưng nguyên liệu chỉ vào được qua một cửa duy nhất với lưu lượng cố định. **Cường độ số học** chính là số thao tác nhà máy làm được trên mỗi kilogram nguyên liệu nhập vào. Nếu công đoạn của bạn làm nhiều việc trên mỗi kilogram — như nhân hai ma trận lớn — thì thợ bận rộn và cửa nhập hàng thành thơi. Nếu nó chỉ làm một hai phép trên mỗi kilogram — như **BatchNorm** hay hàm kích hoạt — thì thợ đứng chơi và cả nhà máy chạy đúng bằng

tốc độ cửa. Giảm số việc phải làm trong trường hợp thứ hai không rút ngắn được gì cả, vì nút thất nằm ở cửa. Đây là toàn bộ nội dung của mô hình *roofline*, và cũng là lý do **hợp nhất kernel** (*kernel fusion*) — gộp nhiều công đoạn để nguyên liệu chỉ phải vào cửa một lần — lại là biện pháp mạnh đến vậy.

1.2 Roofline và cường độ số học (Roofline and Arithmetic Intensity)

A. Một công thức và một hằng số

Với một phép toán bất kỳ, định nghĩa **cường độ số học** (*arithmetic intensity*) là

$$I = \frac{\text{số FLOP thực hiện}}{\text{số byte phải chuyển giữa bộ nhớ và chip}}.$$

Nói gọn thì I đo mức độ “đáng công vận chuyển” của một phép toán. Hiệu năng đạt được bị chặn bởi hai thứ cùng lúc — đỉnh tính toán của chip và băng thông nhân với cường độ — nên [1]

$$P_{\text{đạt được}} = \min(P_{\text{đỉnh}}, B \cdot I).$$

Câu diễn giải: nếu mỗi byte không mang theo đủ việc để làm, chip sẽ chạy theo nhịp của bộ nhớ chứ không theo nhịp của chính nó. Chỗ hai chặn cắt nhau, $I^* = P_{\text{đỉnh}}/B$, là **điểm gãy** (*ridge point*): bên trái nó bạn nghẽn bộ nhớ, bên phải bạn nghẽn tính toán. Điểm gãy là một hằng số của *máy*, không phải của mô hình, và nó đáng thuộc lòng cho từng thiết bị bạn dùng.

- **NVIDIA A100 80 GB** — đỉnh khoảng 312 TFLOP/s fp16 (lỗi tensor, dày), băng thông HBM2e 2039 GB/s. Điểm gãy ≈ 153 FLOP/byte.
- **Jetson AGX Orin 64 GB** — đỉnh khoảng 68 TFLOP/s fp16 (suy ra từ con số công bố 275 TOPS INT8 thừa; tôi chưa tự đo), băng thông LPDDR5 204,8 GB/s. Điểm gãy ≈ 332 FLOP/byte.

Con số thứ hai là điều đáng ngạc nhiên nhất của cả chương: *điểm gãy của thiết bị nhúng cao gấp đôi máy chủ*, vì bộ nhớ LPDDR chậm hơn nhiều so với đà tăng của đơn vị tính toán. Hệ quả thực hành rất nặng: một tối ưu chứng minh được là hữu ích trên máy chủ có thể vô ích trên Orin, và ngược lại, kernel fusion trên Orin thường đáng giá hơn nhiều so với việc cắt FLOPs.

B. Ví dụ nhỏ nhất tính tay được

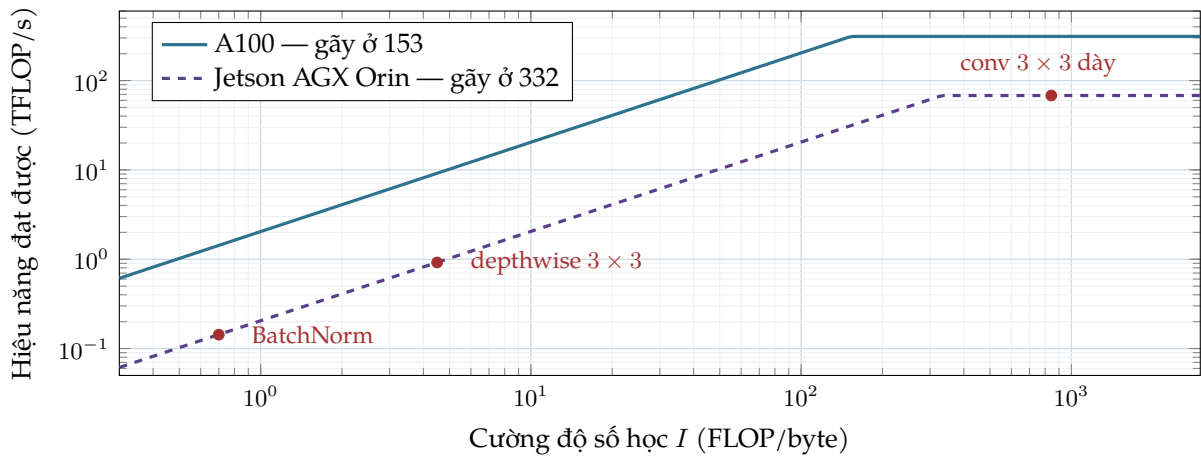
Lấy một *feature map* $256 \times 56 \times 56$ ở fp16 (2 byte mỗi phần tử), tức 802 816 phần tử và 1,61 MB mỗi lần đọc hoặc ghi. So sánh ba phép toán chạy trên chính nó, batch bằng 1.

- **Convolution 3×3 dày, 256 $\rightarrow$ 256 kênh.**
 - FLOPs = $2 \cdot 9 \cdot 256 \cdot 256 \cdot 56 \cdot 56 \approx 3,70 \cdot 10^9$.
 - Byte = vào 1,61 MB + ra 1,61 MB + trọng số $9 \cdot 256 \cdot 256 \cdot 2 = 1,18 \text{ MB} \approx 4,4 \text{ MB}$.
 - $I \approx 843$ FLOP/byte — nằm bên phải điểm gãy của *cả hai* máy, tức compute-bound thật sự.
- **BatchNorm trên cùng tensor đó.**
 - FLOPs ≈ 4 phép trên mỗi phần tử $\approx 3,2 \cdot 10^6$; byte ≈ 3 lượt qua tensor $\approx 4,8 \text{ MB}$.
 - $I \approx 0,7$ FLOP/byte — thấp hơn convolution hơn một nghìn lần. Đây là câu trả lời cho câu hỏi “vì sao BatchNorm memory-bound còn conv thì compute-bound”, và cũng là lý do mọi trình biên dịch đều cố gộp BatchNorm vào conv liền trước nó.
- **Convolution 3×3 tách theo chiều sâu (depthwise), 256 kênh.**

- FLOPs = $2 \cdot 9 \cdot 256 \cdot 56 \cdot 56 \approx 1,45 \cdot 10^7$ — ít hơn bản dày đúng 256 lần.
- Byte $\approx 3,2$ MB — chỉ ít hơn bản dày 1,4 lần.
- $I \approx 4,5$ FLOP/byte. Nằm sâu bên trái điểm gãy, nên thời gian chạy quyết định bởi 3,2 MB kia chứ không bởi 14 triệu phép tính.

Ba dòng trên đủ để trả lời câu hỏi mở đầu chương, và đáng làm cho tới cùng. Hãy thay một convolution dày 3×3 bằng cặp *depthwise* 3×3 cộng *pointwise* 1×1 , đúng công thức của MobileNet [2]. Tổng phép tính là $1,45 \cdot 10^7 + 4,11 \cdot 10^8 \approx 4,3 \cdot 10^8$ FLOP, tức ít hơn 8,7 lần. Nhưng tổng lưu lượng bộ nhớ lại là $3,2 + 3,3 \approx 6,5$ MB, nhiều hơn 1,5 lần so với 4,4 MB của bản dày. Lý do là bây giờ có hai kernel, nên tensor trung gian phải ghi ra rồi đọc lại. Trên một máy nghèo bộ nhớ, thời gian tỉ lệ với byte chứ không với FLOP, nên phép “tối ưu” này làm mô hình *chậm đi* khoảng một rưỡi lần trong khi bảng số liệu khoe giảm FLOPs gần chín lần. Không có gì mâu thuẫn ở đây cả; chỉ là hai đại lượng khác nhau bị gọi chung là “chi phí”.

Hình 1.1 đặt cả ba phép toán lên mái nhà roofline của hai máy để thấy vị trí tương đối của chúng.



Hình 1.1. Roofline của hai máy và vị trí ba phép toán (chấm đặt trên mái của Orin). Phần dốc bên trái là vùng nghèo băng thông, phần phẳng bên phải là vùng nghèo tính toán. Điểm gãy của Orin nằm xa bên phải hơn A100, nên trên thiết bị những có *nhiều* phép toán rơi vào vùng nghèo bộ nhớ hơn. Các đỉnh lấy từ thông số công bố, không phải đo thực tế.

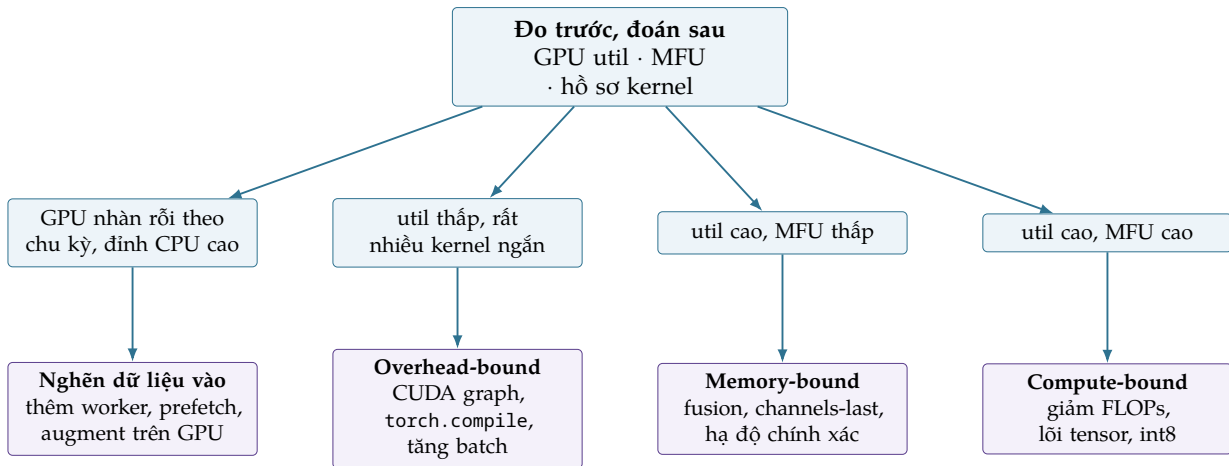
C. Ba chế độ nghèo và biện pháp tương ứng

Roofline mới chỉ phân biệt hai chế độ. Thực tế có chế độ thứ ba, và nó là chế độ phổ biến nhất ở suy luận batch nhỏ trên thiết bị nhúng: GPU không chờ tính toán cũng không chờ bộ nhớ mà chờ CPU gửi việc cho nó. Bảng 1.1 là bảng đáng dán lên tường.

Bảng 1.1. Ba chế độ nghèo: triệu chứng, biện pháp đúng, và biện pháp hoàn toàn vô ích.

Chế độ	Triệu chứng đo được	Biện pháp đúng	Vô ích
Compute-bound	GPU util cao, MFU cao ($> 50\%$)	Giảm FLOPs thật, dùng lõi tensor, lượng tử hoá xuống int8	Kernel fusion
Memory-bound	GPU util cao nhưng MFU thấp; thời gian tỉ lệ với kích thước tensor	Kernel fusion, bố cục <i>channels-last</i> , giảm số lượt đọc/ghi, hạ độ chính xác để giảm byte	Giảm FLOPs
Overhead-bound	GPU util thấp, hồ sơ đầy kernel ngắn dưới $10 \mu s$, khoảng trống giữa các kernel	<i>CUDA graph</i> , <i>torch.compile</i> , gộp kernel, tăng batch	Đổi kiến trúc, giảm FLOPs

Hình 1.2 biến bảng đó thành một quy trình chẩn đoán theo thứ tự, kèm một nhánh thứ tư mà người làm thị giác gặp thường xuyên hơn cả ba nhánh kia cộng lại.



Hình 1.2. Cây chẩn đoán: bốn nhánh, và điều quan trọng nhất là biện pháp của nhánh này gần như vô dụng ở nhánh kia. Nhánh trái ngoài cùng — nghẽn ở đường dữ liệu vào — là nhánh đặc thù của thị giác máy tính và hay bị bỏ sót nhất.

1.3 Cùng FLOPs, khác nhiều lần độ trễ (Equal FLOPs, Unequal Latency)

Phần trước đã cho một nguyên nhân. Còn ba nguyên nhân nữa, và cộng lại chúng giải thích được gần hết các trường hợp dự đoán sai.

1. **Cường độ số học khác nhau** — đã tính ở trên: hai mạng cùng FLOPs có thể chênh nhau hai bậc về lưu lượng bộ nhớ. Đây là nguyên nhân lớn nhất và cũng dễ ước lượng nhất bằng giấy bút.
2. **Số lượng kernel và chi phí phóng kernel** — mỗi lần phóng một kernel tốn cỡ 5–10 μs trên máy chủ và nhiều hơn trên nền CPU yếu của thiết bị nhúng (đây là bậc độ lớn thường được đo, không phải một con số chuẩn). Một mạng nhẹ kiểu MobileNet có thể có hai trăm kernel ở batch 1, tức một sàn cỡ 1–2 ms mà không lượng FLOPs nào ảnh hưởng tới. Với ngân sách 33 ms cho cả pipeline, cái sàn đó không hề nhỏ.
3. **Bố cục bộ nhớ và điều kiện dùng lỗi tensor** — lỗi tensor chỉ được kích hoạt khi tensor ở đúng bố cục (*NHWC*, tức *channels-last*) và khi số kênh chia hết cho một bội số nhất định (8 với fp16). Sai một trong hai điều kiện thì kernel rơi xuống đường chậm và mất nhiều lần hiệu năng *trong khi FLOPs không đổi một chút nào*. Một mạng có số kênh 100 thay vì 96 hoặc 104 có thể chậm hơn đáng kể mà biểu đồ FLOPs không hé lộ gì.
4. **Mức độ song song có sẵn** — đây là chỗ nổi thẳng về Bảng ba trực ở chương 10

↪ **hệ quả của** B/10 — bảng ba trực ở chương kiến trúc giờ đọc được bằng đơn vị thời gian thay vì bằng đơn vị FLOPs.

: hai mạng cùng FLOPs, một sâu và hẹp, một nông và rộng, có độ trễ rất khác nhau ở batch 1 vì các tầng phải chạy nối tiếp còn các kênh thì trải ra được trên hàng nghìn lõi. Ở batch 64 thì khoảng cách này thu hẹp lại, vì batch cung cấp nguồn song song thay thế.

Điểm chung của cả bốn: *FLOPs là một đại lượng của mô hình, độ trễ là một đại lượng của cặp mô hình–máy*. Không có gì bảo đảm hai đại lượng đó tỉ lệ với nhau, và trong họ mạng nhẹ chúng thường xuyên đi ngược nhau.

1.4 Bộ nhớ: kích hoạt so với tham số (Memory: Activations vs Parameters)

A. Vì sao mô hình vừa lúc suy luận lại tràn lúc huấn luyện

Đây là câu hỏi ai cũng gặp trong tuần đầu tiên. Bộ nhớ lúc huấn luyện gồm bốn phần rất khác nhau về cách chúng lớn lên:

- **Tham số** — cố định, không phụ thuộc batch. N tham số ở fp32 tốn $4N$ byte.
- **Gradient** — cùng kích thước với tham số, cũng cố định.
- **Trạng thái optimizer** — với Adam có hai moment nên thêm $8N$ byte. Cộng cả ba: **16 byte cho mỗi tham số**, một con số đáng thuộc.
- **Kích hoạt (activation)** — các tensor trung gian phải giữ lại để dùng trong lượt ngược. Đây là phần *tỉ lệ với batch và với bình phương độ phân giải*, và nó là phần gây tràn.

Ba phần đầu là lý do một mô hình 25 triệu tham số cần khoảng 410 MB chỉ để tồn tại trong vòng huấn luyện. Phần thứ tư là lý do cùng mô hình đó tràn 24 GB khi bạn nâng batch hoặc độ phân giải.

B. Tính tay bộ nhớ kích hoạt của ResNet-50

Hãy làm thật với ResNet-50 ở 224×224 , fp16. Đếm số phần tử của các đầu ra convolution cho một ảnh, theo từng tầng ($56^2 = 3136$, $28^2 = 784$, $14^2 = 196$, $7^2 = 49$):

- **Stem**: $64 \cdot 112^2 = 0,80$ M, sau max-pool $64 \cdot 3136 = 0,20$ M.
- **Stage 1** (3 khối ở 56×56 , kênh $64/64/256$): mỗi khối $(64+64+256) \cdot 3136 = 1,20$ M $\Rightarrow$ 3,61 M, cộng nhánh chiều 0,80 M.
- **Stage 2** (4 khối ở 28×28 , $128/128/512$): mỗi khối $768 \cdot 784 = 0,60$ M $\Rightarrow$ 2,41 M.
- **Stage 3** (6 khối ở 14×14 , $256/256/1024$): mỗi khối $1536 \cdot 196 = 0,30$ M $\Rightarrow$ 1,81 M.
- **Stage 4** (3 khối ở 7×7 , $512/512/2048$): mỗi khối $3072 \cdot 49 = 0,15$ M $\Rightarrow$ 0,45 M.

Tổng $\approx 10,1$ triệu phần tử mỗi ảnh, tức 20,2 MB ở fp16. Với batch 32: ≈ 650 MB — và đó mới chỉ là đầu ra của các tầng convolution. Trong thực tế còn phải giữ đầu ra của BatchNorm, mặt nạ của ReLU, các tensor của nhánh cộng; hệ số nhân thực nghiệm thường rơi vào khoảng hai tới ba lần, cho $\approx 1,3$ –2 GB. Đối chiếu với 410 MB của phía tham số: ở batch 32, kích hoạt chiếm gấp ba tới năm lần, và mọi biện pháp tiết kiệm bộ nhớ phải nhắm vào nó. (Hệ số hai–ba lần là kinh nghiệm thực hành, không phải con số tôi đo được cho đúng mô hình này.)

Hai quan sát rơi ra ngay từ phép tính này. Thứ nhất, *tầng chiếm nhiều kích hoạt nhất là tầng đầu chứ không phải tầng có nhiều tham số nhất*: stem một mình đã bằng gần một nửa cả stage 4 cộng lại, trong khi tham số thì tập trung gần hết ở stage 3 và 4. Thứ hai, kích hoạt tỉ lệ với $H \cdot W$. Nâng độ phân giải từ 224 lên 448 do đó nhân bộ nhớ kích hoạt lên bốn lần, trong khi tham số không đổi một byte. Đó chính là cột thứ ba của bảng ba trục ở chương 10, giờ đã thành con số.

C. Đổi tính toán lấy bộ nhớ

Khi kích hoạt là nút thắt, có một cách đổi rất sạch: *gradient checkpointing* — chỉ lưu một số điểm mốc trên đồ thị và tính lại phần ở giữa trong lượt ngược [3]. Đặt các mốc cách nhau $\sqrt{N}$ tầng thì bộ nhớ giảm từ $O(N)$ xuống $O(\sqrt{N})$. Cái giá là một lượt xuôi phụ, cỡ +30% thời gian huấn luyện. Ba biện pháp cùng họ nhắm ba phần bộ nhớ khác nhau, nên phải biết mình đang tràn vì phần nào rồi mới chọn:

- **Gradient checkpointing** — cắt phần kích hoạt, trả bằng thời gian tính lại [3].

- **Độ chính xác hỗn hợp** — cắt một nửa số byte của mỗi kích hoạt [4].
- **Phân mảnh trạng thái optimizer** — cắt phần $16N$ byte của tham số, gradient và hai moment, bằng cách chia chúng qua nhiều thiết bị [5].

→ **ràng buộc bởi** $C/12$ — bộ nhớ kích hoạt chặn batch tối đa, mà batch lại thay đổi động học của tối ưu — nên một ràng buộc phần cứng đi thẳng vào chất lượng mô hình.

I 1.5 Độ trễ, thông lượng và ngân sách thời gian thực (Latency, Throughput, Real-Time Budgets)

Hai mục tiêu này xung đột nhau và hầu hết tranh cãi về “mô hình nào tốt hơn” thực ra là tranh cãi về việc đang tối ưu cái nào. **Thông lượng** (*throughput*) là số mẫu xử lý được mỗi giây khi bạn được phép gom batch lớn; **độ trễ** (*latency*) là thời gian từ lúc một khung hình tới lúc có kết quả cho *chính* khung đó. Tăng batch gần như luôn cải thiện thông lượng — vì nó nâng cường độ số học và pha loãng chi phí phóng kernel — và gần như luôn làm xấu độ trễ, vì khung đầu tiên phải chờ khung cuối cùng của batch. Cấu hình tối ưu cho huấn luyện do đó thường là cấu hình tệ nhất cho robot.

Với hệ thống thời gian thực, cách làm đúng là *chia ngân sách mili-giây trước khi thiết kế*, giống hệt cách bạn phân bổ ngân sách cho các khối của một pipeline SLAM.

→ **trường hợp riêng của** $B/03$ — ngân sách mili-giây là một ràng buộc cứng, và ràng buộc cứng loại bỏ phương án trước khi độ chính xác kịp lên tiếng.

Bảng 1.2 là một cách chia ví dụ cho 30 Hz; con số cụ thể tùy hệ thống, nhưng kỷ luật thì không đổi: mỗi khối có một trần, và khối nào vượt trần phải bị cắt chứ không được “mượn” của khối khác.

Bảng 1.2. Một cách chia ngân sách 33 ms cho pipeline nhận thức 30 Hz (ví dụ minh họa, không phải số đo).

Khối	Trần (ms)	Điều xảy ra khi vượt trần
Thu ảnh + ISP	4	Không cắt được; là ràng buộc cứng của cảm biến
Tiền xử lý, chuyển bố cục	2	Gộp vào kernel đầu của mạng, hoặc làm trên GPU
Backbone	12	Trục đầu tiên bị cắt: hạ độ phân giải hoặc đổi backbone
Head + hậu xử lý	7	NMS trên CPU là thủ phạm quen thuộc
Hợp nhất, quyết định	3	Thường bị quên trong lúc đo mô hình
Dự phòng (jitter, throttling)	5	Không có nó thì hệ thống đúng 90% thời gian

Dòng cuối cùng đáng nói riêng. Trên thiết bị nhúng, năng lượng là ràng buộc thứ nhất chứ không phải thứ hai, và **giảm xung do nhiệt** (*thermal throttling*) khiến mọi phép đo ngắn nói dối: mô hình đo được 25 ms trong ba mươi giây đầu có thể thành 40 ms sau mười phút chạy liên tục trong vỏ kín. Đây là kinh nghiệm thực hành phổ biến trên Jetson và là lý do benchmark phải chạy đủ dài để nhiệt bão hòa.

→ **ràng buộc bởi** $E/24$ — một phép đo tốc độ là một thí nghiệm, nên nó chịu đúng các yêu cầu về làm nóng, lặp lại và báo cáo phân vị.

Nó cũng là một phần lời giải thích vì sao nhiều phương pháp dẫn đầu bảng xếp hạng không dùng được ngoài đời: bảng xếp hạng đo trên máy chủ có tản nhiệt và điện không giới hạn.

1.6 Nút cổ chai không nằm ở GPU (Bottlenecks Outside the GPU)

Đặc thù của thị giác máy tính so với xử lý ngôn ngữ: đầu vào rất nặng và phải được giải mã. Giải một ảnh JPEG 1080p trên một lõi CPU tốn cỡ 5–15 ms, và augmentation còn tốn thêm. Với tám worker, bạn cấp được cỡ vài trăm tới hơn một nghìn ảnh mỗi giây — trong khi một GPU máy chủ nuốt ResNet-50 ở tốc độ cao hơn thế. Kết quả là GPU nhàn rỗi theo chu kỳ, và mọi giờ bạn bỏ ra tối ưu mô hình đều không thay đổi gì. Các lỗi thoát theo thứ tự chi phí tăng dần: tăng số worker và bật prefetch; chuyển augmentation lên GPU; chuyển sang định dạng dữ liệu tuần tự đọc theo khối thay vì hàng triệu file nhỏ; giải mã JPEG bằng phần cứng.

Một nút cổ chai họ hàng là đường truyền giữa host và thiết bị. Một batch 32 ảnh $3 \times 224 \times 224$ ở fp32 là 19 MB. Qua PCIe 4.0 x16 với thông lượng thực tế cỡ 25 GB/s, phép sao chép tốn khoảng 0,8 ms. Con số đó không lớn, nhưng nếu bạn quên dùng bộ nhớ ghim và sao chép bất đồng bộ thì nó nằm trọn trên đường tới hạn. Trên Jetson thì câu chuyện khác hẳn và theo hướng có lợi: CPU và GPU dùng chung bộ nhớ vật lý, nên bản sao đó có thể biến mất hoàn toàn nếu bạn cấp phát đúng loại bộ nhớ — một tối ưu không tồn tại trên máy chủ.

↔ **hệ quả của D/21** — đây đúng là loại tối ưu mà runtime triển khai khai thác, và là lý do cùng một mô hình phải được đo lại trên từng nền tảng đích.

1.7 Các con số cần thuộc lòng (Numbers Worth Memorizing)

Đây là phần duy nhất của chương thực sự là một danh sách. Nó đáng thuộc vì nó cho phép ước lượng bậc độ lớn ngay trong đầu, giữa một cuộc họp.

- **Kích thước dữ liệu.** fp32 = 4 byte, fp16/bf16 = 2, int8 = 1. Một triệu tham số fp32 = 4 MB; huấn luyện với Adam = 16 MB.
- **Công thức chi phí.** Tích chập: $FLOPs = 2k^2 C_{in} C_{out} H_{out} W_{out}$ cho mỗi ảnh. Tầng nối đầy đủ: $2 C_{in} C_{out}$. Activation tỉ lệ với $batch \times H \times W$ — nhân đôi độ phân giải là nhân bốn bộ nhớ.
- **Hằng số của máy.** Điểm gãy roofline của phần cứng hiện đại nằm trong khoảng 100–350 FLOP/byte; đọc 1 GB từ bộ nhớ tốn 0,5 ms ở 2 TB/s và 5 ms ở 200 GB/s; phóng một kernel tốn cỡ 5 μs ; PCIe 4.0 x16 cho cỡ 25 GB/s thực tế.
- **Ngưỡng chẩn đoán.** MFU dưới 30% thì gần như chắc chắn bạn *không* compute-bound, nên đừng đụng vào FLOPs. Kernel ngắn hơn 10 μs là dấu hiệu overhead-bound.
- **Ngân sách thời gian.** 30 Hz = 33 ms mỗi khung; 60 Hz = 16,7 ms; một ảnh JPEG 1080p mất 5–15 ms để giải mã trên một lõi.

1.8 Thực hành

Repo và tài nguyên. torch.profiler kèm trace viewer của TensorBoard là điểm khởi đầu; NVIDIA Nsight Systems cho bức tranh toàn hệ thống và Nsight Compute cho từng kernel; [XuehaiPan/nvtop](#) để nhìn nhanh; [facebookresearch/fvcore](#) để đếm FLOPs; [triton-lang/triton](#) khi bạn cần tự viết kernel hợp nhất. Bài đọc ngắn gọn nhất về ba chế độ nghề là “Making Deep Learning Go Brrrr From First Principles” của Horace He. Danh sách này chốt tại tháng 9/2026; khác với các chương về mô hình, phần lớn nội dung ở đây dựa trên vật lý phần cứng nên bán rã dài hơn nhiều — chỉ các con số tuyệt đối của từng đời chip là sẽ cũ đi.

Câu hỏi kiểm tra hiểu. Mô hình của bạn đạt 30 % MFU: nghẽn ở đâu, và ba phép đo nào phân biệt được ba khả năng? Vì sao BatchNorm memory-bound trong khi convolution 3×3 compute-bound, và điều đó gợi ý gì về việc nên hợp nhất cái gì với cái gì? Nếu chuyển từ A100 sang Orin, những tối ưu nào bạn đang có sẽ mất tác dụng và những tối ưu nào trở nên đáng giá hơn?

Mini project. Hồ sơ hoá một vòng huấn luyện thật và lập bảng thời gian ở năm chỗ: dataloader, lượt xuôi, lượt ngược, bước optimizer, và đồng bộ. Yêu cầu bắt buộc: *viết dự đoán ra giấy trước khi mở profiler*, rồi ghi lại mức sai của từng dự đoán. Bản ghi mức sai đó có giá trị hơn bảng số liệu.

1.9 Giới hạn và trường hợp hỏng

Roofline là một mô hình *thượng giới hạn* và nó nói dối theo những cách có thể liệt kê được. Nó giả định phép toán chồng lấn hoàn hảo giữa tính toán và truyền dữ liệu, giả định cache không tồn tại hoặc luôn trúng, và giả định chỉ có một nút thất tại một thời điểm. Trong thực tế một kernel có thể vừa nghẽn băng thông vừa nghẽn số thanh ghi, và một mô hình có thể compute-bound ở tầng này nhưng overhead-bound ở tầng khác trong cùng một lượt. Roofline dùng để *chọn hướng điều tra*, không dùng để dự đoán con số.

Ba trường hợp hỏng cụ thể đáng cảnh giác. Thứ nhất, *đỉnh công bố không phải đỉnh đạt được*: các con số marketing thường tính ở chế độ thừa hoặc ở độ chính xác thấp nhất, và khoảng cách với thực tế có thể là hai lần — vì vậy hãy đo đỉnh của chính máy mình một lần rồi dùng con số đó mãi. Thứ hai, *benchmark ngắn nói dối* vì nhiệt chưa bão hoà, vì xung chưa ổn định, và vì lần chạy đầu tiên còn gánh chi phí biên dịch kernel; phép đo đúng phải có giai đoạn làm nóng và phải lấy phân vị chứ không chỉ lấy trung bình. Thứ ba, *đo sai chỗ*: lệnh gọi GPU là bất đồng bộ, nên đo bằng đồng hồ của CPU mà quên đồng bộ hoá sẽ cho ra những con số nhanh đến phi lý — đây là lỗi phổ biến nhất trong toàn bộ lĩnh vực đo hiệu năng học sâu.

CẠM BÃY

Trực giác về hiệu năng gần như luôn sai cho tới khi đo, kể cả trực giác của người có kinh nghiệm. Nhưng “cứ đo đi” là lời khuyên chưa đủ, vì đo mà không dự đoán trước thì bạn chỉ thu được một con số chứ không cải thiện được trực giác. Quy trình đúng có ba bước và bước đầu là bước hay bị bỏ: *viết dự đoán ra trước*, đo, rồi ghi lại mức sai. Sau mười lần như vậy bạn sẽ biết mình có thiên kiến hệ thống ở đâu — hầu hết mọi người đánh giá thấp chi phí bộ nhớ và đánh giá cao ảnh hưởng của FLOPs.

CẦN NHỚ

Một câu để mang theo: *FLOPs là thuộc tính của mô hình, thời gian là thuộc tính của cặp mô hình–máy, và cây cầu giữa hai thứ đó là cường độ số học*. Trước khi tối ưu bất cứ điều gì, hãy tính I cho khối bạn định sửa. Rồi đem nó so với điểm gãy của máy bạn đang chạy. Nếu I nằm bên trái điểm gãy, mọi nỗ lực giảm FLOPs sẽ không đem lại gì. [T2]

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● FLOPs	Số phép nhân–cộng dấu phẩy động cho một lần chạy. Nó đo khối lượng công việc tính toán, không đo thời gian, và đó là toàn bộ nội dung của chương.
●●● Cường độ số học	Số FLOP làm được trên mỗi byte phải chuyển giữa bộ nhớ và chip. Nó trả lời câu “mỗi byte kéo về được dùng để làm bao nhiêu việc”, và là đại lượng duy nhất phân biệt nghẽn tính toán với nghẽn bộ nhớ.
●●● Điểm gãy roofline	Đỉnh tính toán chia cho băng thông, một hằng số của <i>máy</i> chứ không của mô hình. Khoảng 150 FLOP/byte trên A100 và khoảng 330 trên Jetson Orin; bên trái nó bạn nghẽn bộ nhớ, bên phải nghẽn tính toán.
●● Vì sao định fp16 công bố không dùng thẳng được?	Vì con số marketing thường tính ở chế độ thừa hoặc ở độ chính xác thấp nhất, và khoảng cách với thực tế có thể tới hai lần. Hãy đo định của chính máy mình một lần bằng một GEMM lớn, rồi dùng con số đó mãi.
●●● Vì sao BatchNorm memory-bound còn convolution compute-bound?	Vì cường độ số học chênh hơn một nghìn lần: convolution 3×3 dày ở 256 kênh cho khoảng 843 FLOP/byte, còn BatchNorm trên cùng tensor chỉ khoảng 0,7. Đó là lý do trình biên dịch luôn cố gộp BatchNorm vào convolution liền trước.
●●● Memory-bound	Chế độ mà thời gian chạy do lưu lượng bộ nhớ quyết định chứ không do số phép tính. Triệu chứng là GPU util cao nhưng MFU thấp; chứa bằng hợp nhất kernel và bố cục channels-last, còn giảm FLOPs thì vô ích.
●● Overhead-bound	Chế độ mà GPU chờ CPU gửi việc. Triệu chứng là util thấp với rất nhiều kernel ngắn dưới $10 \mu s$; chứa bằng CUDA graph, torch.compile hoặc tăng batch, còn đổi kiến trúc thì vô ích.
●●● Vì sao depthwise cộng pointwise giảm FLOPs mà không nhanh hơn?	Vì nó giảm FLOPs 8,7 lần nhưng <i>tăng</i> lưu lượng bộ nhớ từ 4,4 MB lên 6,5 MB: hai kernel nên tensor trung gian phải ghi ra rồi đọc lại. Trên máy nghẽn bộ nhớ, thời gian đi theo byte.
●● Channels-last (NHWC)	Bố cục xếp kênh liền nhau trong bộ nhớ. Lỗi tensor chỉ được kích hoạt khi tensor ở đúng bố cục này và số kênh chia hết cho 8 ở fp16; sai một điều kiện thì kernel rơi xuống đường chậm dù FLOPs không đổi.
●● Chi phí phóng kernel	Mỗi lần CPU gửi một kernel cho GPU tốn cỡ $5\text{--}10 \mu s$, và nhiều hơn trên CPU yếu của thiết bị nhúng. Một mạng nhẹ hai trăm kernel do đó có sần thời gian 1–2 ms mà không lượng FLOPs nào ảnh hưởng tới.
●●● Bộ nhớ kích hoạt	Các tensor trung gian của lượt xuôi, phải giữ lại vì lượt ngược cần tới. Nó tỉ lệ với batch và với bình phương độ phân giải, nên nó mới là thứ chặn batch chứ không phải số trọng số.
●● Trạng thái optimizer	Với Adam là hai moment, cộng với trọng số và gradient thành 16 byte cho mỗi tham số ở fp32. Phần này cố định, không phụ thuộc batch, nên nó chỉ phổ biến khi mô hình rất lớn mà rất nông.
●● Gradient checkpointing	Chỉ lưu một số điểm mốc trên đồ thị rồi tính lại phần ở giữa trong lượt ngược. Đặt mốc cách nhau $\sqrt{N}$ tầng thì bộ nhớ giảm từ $O(N)$ xuống $O(\sqrt{N})$, trả bằng cỡ +30% thời gian huấn luyện.
●● Tăng batch xong thông lượng tăng nhưng độ trễ đuôi xấu hẳn — vì sao?	Vì hai đại lượng này xung đột. Gộp batch nâng cường độ số học và pha loãng chi phí phóng kernel, nên thông lượng tăng; nhưng khung hình đầu phải chờ khung cuối của batch, nên độ trễ xấu đi. Cấu hình tốt cho huấn luyện thường là cấu hình tệ nhất cho robot.
●● Ngân sách mili-giây	Chia trần thời gian cho từng khối <i>trước</i> khi thiết kế: 33 ms cho 30 Hz, 16,7 ms cho 60 Hz. Khối nào vượt trần phải bị cắt chứ không được mượn của khối khác, và phải chừa phần dự phòng cho jitter và giảm xung.
● Giảm xung do nhiệt	Thiết bị tự hạ tần số khi nóng, nên một mô hình đo 25 ms trong ba mươi giây đầu có thể thành 40 ms sau mười phút chạy trong vỏ kín. Vì vậy benchmark phải chạy đủ dài để nhiệt bão hoà.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●● Vì sao dataloader có thể là nút cổ chai của cả hệ?	Giải mã một ảnh JPEG 1080p tốn 5–15 ms trên một lõi CPU, nên tám worker chỉ cấp được vài trăm tới hơn một nghìn ảnh mỗi giây. Con số đó thấp hơn tốc độ nuốt của một GPU máy chủ, và mọi tối ưu mô hình khi đó đều vô nghĩa.
●●● GPU util 95 % nhưng MFU chỉ 12 % — nghĩa là gì?	Bạn memory-bound: GPU bận, nhưng nó bận chờ bộ nhớ chứ không bận tính. Phân biệt với compute-bound bằng chính MFU, và phân biệt với overhead-bound bằng util — overhead-bound thì util thấp chứ không cao.
●● GPU nhàn rỗi theo chu kỳ, đỉnh CPU cao — nguyên nhân?	Nghẽn ở đường dữ liệu vào, không ở mô hình. Thêm worker và bật prefetch rồi đo lại; nếu thời gian một bước giảm theo số worker thì đúng là nhánh này.
●●● Lượt xuôi của một mạng lớn đo được 0,2 ms — lỗi ở đâu?	Bạn quên đồng bộ hoá trước khi bấm giờ. Lệnh gọi GPU là bất đồng bộ nên đồng hồ CPU trả về ngay khi lệnh được xếp hàng, chứ không phải khi phép tính xong.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
FlashAttention	Hệ quả trực tiếp của phân tích roofline: nó không giảm FLOPs mà giảm số lượt đọc/ghi bộ nhớ — đúng biện pháp của chế độ memory-bound	Ví dụ sạch nhất trong ngành về việc đo đúng đại lượng [6]
torch.compile, TensorRT, ONNX Runtime	Hợp nhất kernel và ghi lại đồ thị thành CUDA graph — biện pháp của chế độ memory-bound và overhead-bound	Hạ tầng bền; là nơi phần lớn tốc độ “miễn phí” đến từ
NVIDIA DALI và các định dạng dữ liệu tuần tự	Tồn tại vì nút cổ chai giải mã ảnh trên CPU, không vì bất kỳ lý do nào liên quan tới mô hình	Đúng nhánh trái ngoài cùng của cây chẩn đoán trong chương
Gradient checkpointing trong PyTorch và Hugging Face	Đổi một lượt xuôi phụ lấy bộ nhớ kích hoạt $O(\sqrt{N})$	Bật bằng một cờ; là lý do nhiều mô hình lớn huấn luyện được trên phần cứng nhỏ [3]
DeepSpeed ZeRO và FSDP	Cắt đúng phần 16 byte mỗi tham số của trạng thái optimizer bằng cách phân mảnh qua nhiều thiết bị	Nhắm một phần bộ nhớ khác hẳn với checkpointing — biết mình tràn vì phần nào mới chọn đúng [5]

MINI PROJECT SLAM · Roofline đo thật cho Jetson Orin của bạn, với ba khối trong pipeline SLAM · 1–2 ngày

Mục tiêu	thay mọi con số công bố trong chương này bằng con số đo được của chính máy bạn. Đáng ngờ nhất là đỉnh fp16: ở đây nó chỉ được <i>suy ra</i> từ thông số 275 TOPS INT8 thừa, chưa hề được đo. Có mái nhà đo thật rồi thì dùng nó để phân loại ba khối trong pipeline của bạn.
Dữ liệu	một chuỗi EuRoC chạy qua pipeline hiện tại; cố định xung bằng jetson_clocks và chạy đủ lâu cho nhiệt bão hoà <i>trước khi</i> bắt đầu ghi số.
Các bước	(1) đo đỉnh tính toán bằng một GEMM fp16 đủ lớn lặp nhiều lần; (2) đo bằng thông thực bằng một phép sao chép tensor lớn hơn hẳn cache; (3) vẽ mái nhà và tính điểm gãy đo được, đặt cạnh con số 332 FLOP/byte suy ra từ thông số; (4) chọn ba khối trong pipeline — ví dụ trích đặc trưng ORB trên GPU, một tầng convolution của backbone, và một bước chuẩn hoá hoặc đổi kích thước theo phần tử — rồi đếm bằng tay số FLOP và số byte của từng khối; (5) đặt ba điểm lên đồ thị và đối chiếu thời gian dự đoán với thời gian đo bằng Nsight Systems.

- Xong khi** bạn có một đồ thị roofline với đỉnh và bằng thông *đo được*, và ba khối của bạn nằm trên đó với toạ độ tính bằng tay. Mỗi khối kèm một câu kết luận dạng “khối này memory-bound ở I FLOP/byte, nên hợp nhất kernel đáng giá còn cắt FLOPs thì không”. Cuối cùng, bạn ghi được sai số giữa thời gian roofline dự đoán và thời gian đo thật.
- Nối với** D/20 và D/21 để biến kết luận thành hành động; E/24 cho cách báo cáo đúng (làm nóng, lặp lại, phân vị thay vì trung bình); B/10 nếu kết luận buộc bạn đổi backbone thay vì đổi kernel.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì FLOPs không đo thứ chi phối thời gian. Hai mô hình cùng FLOPs có thể chênh nhau hai bậc về cường độ số học. Chúng cũng có thể chênh hàng trăm lần về số kernel phải phóng, và khác nhau ở chỗ có kích hoạt được lõi tensor hay không. Ba nguyên nhân này độc lập với nhau, và không cái nào xuất hiện trong con số FLOPs.
- A2. Vì trọng số chỉ là một trong bốn phần bộ nhớ. Huấn luyện thêm gradient, thêm trạng thái optimizer (Adam giữ hai moment, tổng cộng 16 byte mỗi tham số), và quan trọng nhất là kích hoạt phải giữ lại cho lượt ngược — phần duy nhất tỉ lệ với batch và với bình phương độ phân giải.
- A3. Rằng phần lớn danh sách đó sẽ đối thứ tự. Điểm gây cao hơn nghĩa là nhiều phép toán rơi vào vùng nghẽn bộ nhớ hơn, nên hợp nhất kernel và giảm số lượt đọc/ghi trở nên đáng giá hơn, còn cắt FLOPs kém hiệu quả hơn hẳn so với trên máy chủ.
- A4. Vì có một nhánh chẩn đoán thứ tư nằm ngoài GPU: giải mã ảnh và augmentation trên CPU. Một ảnh JPEG 1080p tốn 5–15 ms trên một lõi, nên tám worker chỉ cấp được vài trăm tới hơn một nghìn ảnh mỗi giây — thấp hơn tốc độ nuốt của một GPU máy chủ.
- A5. Vì lệnh gọi GPU là bất đồng bộ: đồng hồ CPU trả về ngay khi lệnh được xếp hàng chứ không phải khi phép tính xong. Không đồng bộ hoá trước khi bấm giờ là lỗi đo phổ biến nhất trong toàn bộ lĩnh vực này.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn gọi thợ sửa ống nước. Việc thật sự chỉ mất năm phút vận cờ lê, nhưng anh ta phải lái xe bốn mươi phút mới tới nhà bạn. Đoán hoá đơn bằng cách đếm số phút vận cờ lê thì bạn sai gần mười lần. Thuê thợ khéo gấp đôi thì hoá đơn gần như không đổi, vì phần lớn thời gian nằm ở đường đi chứ không ở tay nghề.

Máy tính chạy mạng nơ-ron y hệt vậy. Con số mọi người hay khoe, tức số phép tính phải làm, chính là số phút vận cờ lê. Thứ tốn thời gian là chuyển dữ liệu từ bộ nhớ vào chỗ tính, tức quãng đường lái xe. Nên phép đo đáng giá không phải “bao nhiêu việc” mà là “mỗi chuyến đi làm được bao nhiêu việc”.

Nếu mỗi chuyến chở về đủ việc làm cả buổi thì cắt bớt việc mới giúp. Nếu mỗi chuyến chỉ làm được một hai thao tác thì cắt việc chẳng đổi gì, phải gộp nhiều thao tác vào chung một chuyến. Còn nếu xe cứ đứng chờ vì bạn gọi đặt lịch từng việc một, thì phải gộp các cuộc gọi.

Cái giá là bạn buộc phải đo. Trực giác về tốc độ gần như luôn sai, và tỉ lệ giữa thời gian đi với thời gian làm khác nhau ở mỗi máy: kết luận đúng trên máy chủ có thể sai hẳn trên thiết bị nhúng.

Chỗ không rút gọn được: một phép tính có thể vừa kẹt ở đường đi vừa kẹt ở chỗ khác cùng lúc, nên bức tranh “chỉ có đúng một nút thắt” là một xấp xỉ. Nó đủ tốt để chọn hướng điều tra, nhưng không đủ để dự đoán ra con số.

Một câu: Thời gian chạy không được quyết định bởi bạn làm bao nhiêu việc, mà bởi bạn phải đi lấy dữ liệu bao nhiêu lần — nên trước khi cắt việc, hãy đếm số chuyến đi.

1.10 Kết nối

Chương này là *mặt sau* của chương 10: mọi đánh đổi kiến trúc ở đó đều có một cái giá vật lý được định lượng ở đây, và cột “độ trễ” trong bảng ba trục chỉ đọc được sau khi có khái niệm cường độ số học. Nó cũng là *trường hợp cụ thể* của nguyên lý ràng buộc cứng ở chương 3: ngân sách mili-giây và ngân sách năng lượng loại bỏ phương án trước khi độ chính xác kịp lên tiếng, và đó là lý do phần lớn thiết kế trong hệ thống nhúng trông “bảo thủ” hơn trong paper.

Đọc tiếp: D/20-toi-uu-va-nen-mo-hinh-bien-cac-quan-sat-o-dây-thành-hành-động — lượng tử hoá, tia, chưng cất — và mỗi kỹ thuật ở đó nhắm vào một chế độ nghẽn khác nhau, nên chương này là điều kiện để chọn đúng. D/21-inference-va-trien-khai bàn về trình biên dịch và runtime, tức là nơi kernel fusion và CUDA graph thực sự xảy ra. C/12-co-che-huan-luyen giải thích vì sao batch lớn không chỉ là chuyện bộ nhớ mà còn thay đổi động học của tối ưu, nên bạn không được tự do tăng batch để lấy thông lượng. Và E/24-thi-nghiem-va-ablation dạy cách đo cho tử tế — phân vị, lặp lại, làm nóng — những thứ mà phần “giới hạn” ở trên vừa chỉ ra là bắt buộc.

■ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao thay convolution dày bằng depthwise cộng pointwise giảm FLOPs gần chín lần mà vẫn có thể làm mô hình chậm đi, và hai con số byte nào chứng minh điều đó?
2. Điểm gãy roofline của Jetson Orin cao hơn của A100. Điều đó có nghĩa Orin “tốt hơn” ở khía cạnh nào và “tệ hơn” ở khía cạnh nào, và nó đổi hướng tối ưu của bạn ra sao?
3. Mô hình vừa khít GPU khi suy luận nhưng tràn khi huấn luyện với cùng batch: bốn thành phần bộ nhớ nào đang tồn tại, và thành phần nào chịu trách nhiệm?
4. Với ResNet-50 batch 32, vì sao tầng chiếm nhiều kích hoạt nhất lại không phải tầng có nhiều tham số nhất?
5. GPU util của bạn là 95 % nhưng MFU chỉ 12 %. Đó là chế độ nào, và vì sao “giảm FLOPs” là biện pháp sai?
6. Vì sao một benchmark 30 giây trên thiết bị nhúng có thể cho con số tốt hơn thực tế vận hành tới gần hai lần?
7. Bạn đo được lượt xuôi mất 0,2 ms trên một mạng lớn — lỗi đo nào gần như chắc chắn đã xảy ra?

Nguồn

- [1] S. Williams, A. Waterman, and D. Patterson, “Roofline: An Insightful Visual Performance Model for Multicore Architectures,” *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.
- [2] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [3] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [4] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [5] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [6] T. Dao, D. Y. Fu, S. Ermon, et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022.